Network Programming - 16
Network Library 구현

afewhee@gmail.com

- 공통 함수 라이브러리화
- I/O 모델 추상화
- I/O 모델 구체화
- 자료
  - https://github.com/3dapi/dv05_netlib

- 개별적인 네트워크 코드의 문제
  - 유지 보수, 중복 코드, 성능
- 해결 방안
  - 공통 사용 함수 라이브러리화
  - 공통 인터페이스 추상화
  - 모델별 클래스 구체화
- 구현
  - 추상화 → 구체화 → 라이브러리화

- 전체 코드 분석
  - I/O 모델별 분리

- 설계 방향 (추상화)
  - 1단계: 반복 코드 발견
  - 2단계: 공통 함수 분리
  - 3단계: 공통 인터페이스 정의
  - 4단계: 기본 클래스 작성
  - 5단계: I/O 모델별 파생 클래스 구현
  - 6단계: 응용 계층은 ILcNet만 사용

WinSock API 직접 사용

↓

공통 함수 분리

↓

LcNetUtil

↓

ILcNet 추상화

↓

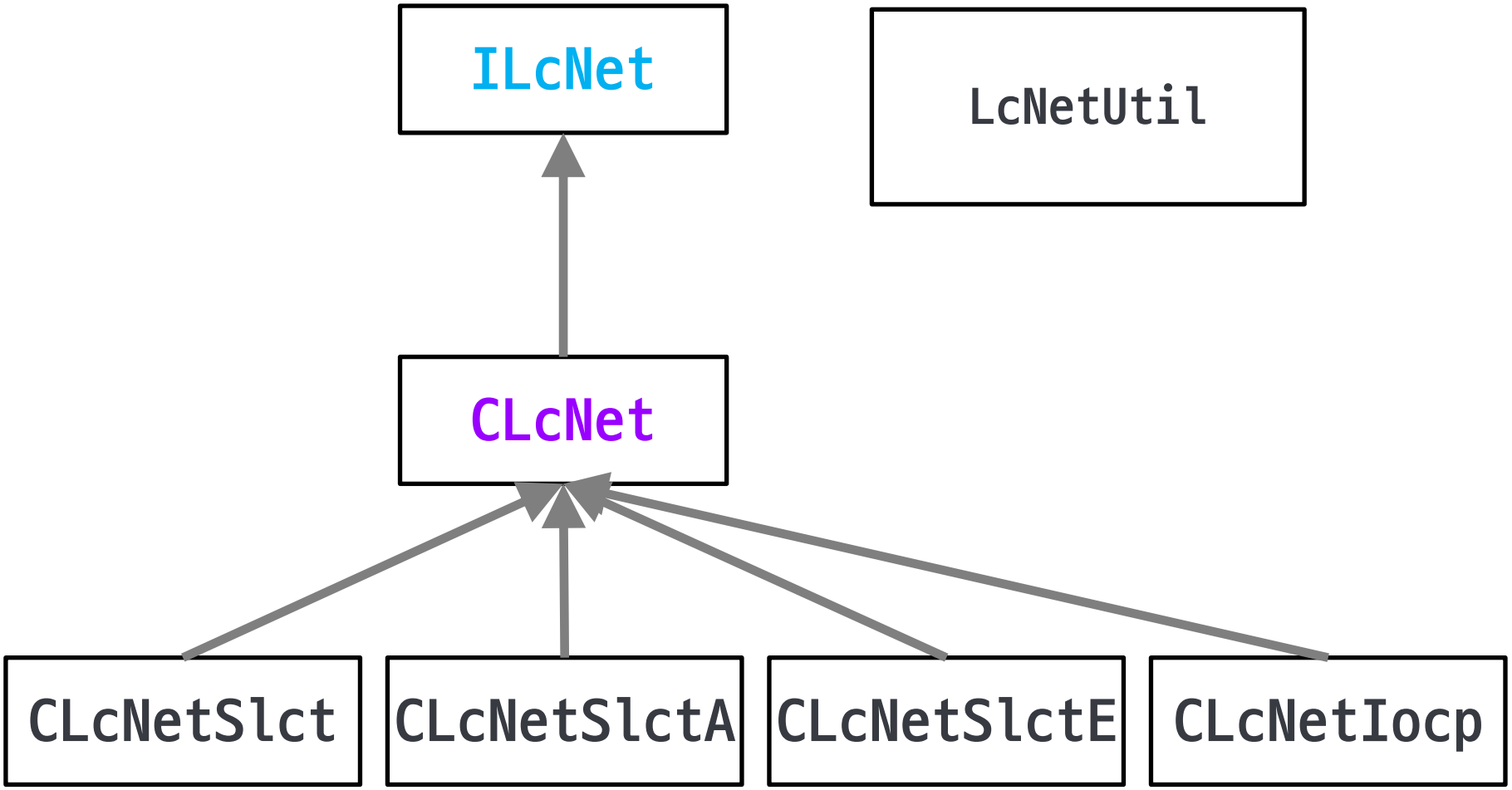
CLcNet 공통 기반

↓

CLcNetSlct / CLcNetSlctA / CLcNetSlctE / CLcNetIocp

↓

응용 계층 코드 단순화



- 설계 방향
  - 분리 → 역할 정리(함수 정리)
  - 클래스 분리 → I/O 모델 교체
  - Send / Recv wrapping → 패킷 처리 규칙 통일
  - FrameMove (Update) → 네트워크 갱신 지점 통일
  - NetUtil → BSD, Linux, MacOS, WinSock API 반복 제거

- 정리
  - 코드 나열 중심 → 설계 진행 과정 중심
  - 파일 나열 중심 → 역할과 책임 중심

- 구현 내용
  - 서버와 클라이언트가 같은 코드를 반복
  - socket / bind / listen / connect / accept 반복
  - send / recv 처리 중복
  - 오류 처리 중복
  - socket option 설정 중복
  - Thread / Event 생성 코드 반복
  - I/O 모델 변경 시 코드 전체 변경

```
Server
├ WSAStartup
├ socket
├ bind
├ listen
├ accept
├ send / recv
└ closesocket
```

```
Client
├ WSAStartup
├ socket
├ connect
├ send / recv
└ closesocket
```

- 설계 방향
  - WinSock 초기화 / 해제
  - 소켓 생성 / 닫기 / 주소 설정
  - Bind / Listen / Accept / Connect
  - Select, NonBlocking
  - Nagle Off
  - Thread 생성 / 종료
  - Event 생성 / 대기
  - Packet Encode / Decode
  - RingBuffer

반복되는 저수준 코드
↓
LcNetUtil
↓
상위 네트워크 클래스에서 공통 사용

1.7 라이브러리화 목표

- 설계 방향
  - 응용 코드에서 WinSock API 제거
  - 서버/클라이언트 코드 단순화
  - 패킷 송수신 규칙 통일
  - I/O 모델 교체 가능
  - 응용 코드와 네트워크 처리 역할 분리
  - 유지보수 가능한 구조

- 응용 계층 사용 목표

```
ILcNet* net = nullptr;
LcNet_Create("Select", &net, ip, port, "TCP", "Server");

net->FrameMove();
net->Send(data, size, MSG_CHAT);
net->Recv(buffer, &recvSize, &msgType);
```

2.1 공통 함수 라이브러리화: LcNetUtil

- LcNetUtil의 역할
  - 구현 클래스가 공통으로 사용하는 기반 함수

- 설계 방향
  - API Wrapping
    - WinSock API
    - Socket API
    - Thread API
    - Event, IOCP API
  - Packet
    - Encode / Decode
  - RingBuffer

2.2 Socket 공통 함수 구성

- 주요 내용
  - 소켓 생성 코드 공통화
  - 주소 설정 공통화
  - 연결 / 바인딩 / 리슨 / Accept 공통화
  - NonBlocking / Nagle Off 공통화

- Blackbox
  - 내부 구현은 보지 않고, 입력과 출력만 기준으로 대상을 이해, 검증하는 방식 → 내부 구현 모름 입력·출력 기준

- 구현 클래스
  - 직접 WinSock API 사용 대신 공통함수를 사용
    → 구현 클래스에 대한 블랙박스화

2.3 Socket 생성 추상화

- 설계 방향
  - TCP / UDP 생성 함수 분리
  - Overlapped 여부를 인자로 처리
  - Select 계열은 일반 소켓
  - IOCP는 Overlapped 소켓 필요
  - 같은 함수 이름으로 생성 정책 통일

- 주요 내용
  - Thread 생성 방식 통일
  - Event 생성 방식 통일
  - I/O 모델별 구현에서 반복 제거
  - AsyncSelect / EventSelect / IOCP에서 사용

- 구현 내용
  - Thread → Create / End / Close / Resume / Suspend / Wait
  - Event → Create / Close / Resume / Suspend / Wait

2.5 WSAEvent / IOCP 공통 함수

- 주요 내용
  - EventSelect 구현을 위한 WSAEvent 래핑
  - IOCP 구현을 위한 Completion Port 래핑
  - API 호출 위치 분산 방지
  - I/O 모델별 클래스는 흐름만 담당

- WSA wrapping 구현 내용
  - WSAEventCreate / WSAEventClose
  - WSAEventSelect
  - WSAEventWaits / WSAEventEnum
  - GetSystemProcessNumber

2.6 Packet / RingBuffer 라이브러리화

- 설계 방향
  - Send / Recv는 문자열 전송이 아니라 패킷 전송
  - 패킷 구조를 모든 모델에서 동일하게 사용
  - rbSnd는 보낼 패킷 보관
  - rbRcv는 받은 패킷 보관
  - 패킷 Encode / Decode를 LcNetUtil로 분리

- 구현 내용
  - Encode / PacketDecode

- 주요 내용
  - 모든 네트워크 모델이 같은 패킷 구조 사용
  - Send는 원본 메시지를 패킷으로 변환
  - Recv는 패킷에서 메시지 복원
  - 메시지 종류 dMsg로 구분
  - TCP 스트림 위에 메시지 경계 생성

2.8 Packet 처리 구조

[응용 계층 Data]
↓
LcNet_PacketEncode
↓
[Length 2][Kind 4][Message N][Tail 4]
↓
rbSnd
↓
send

recv
↓
rbRcv
↓
LcNet_PacketDecode
↓
[응용 계층 Data + Message Kind]

- 처리 과정
  - TCP recv 결과 임시 저장
  - 완성된 패킷 단위 PopFront
  - 헤더 길이 기반 패킷 크기 확인
  - Send 요청을 rbSnd에 보관 후 실제 전송
  - Select / AsyncSelect / EventSelect / IOCP 공통 버퍼

2.10 TRingBuf 처리 위치

- 링 버퍼 push /pop

```
int TRingBuf::PushBack(BYTE* pSrc, int iSize)
{
  if(W < (iSize + S))
    return -1;
  int iLen = iSize;
  while(iLen--)
  {
    *(B + L) = *(pSrc++);
    ++L;
    L %= W;
  }
  S += iSize;
  return 0;
}
```

```
int TRingBuf::PopFront(BYTE* pDst, WORD* iSize)
{
  BYTE sSize[PCK_BUF_HEAD] = {0};
  int T = F;
  WORD iLen = 0;

  *(sSize + 0) = *(B + T);
  ++T;
  T %= W;

  *(sSize + 1) = *(B + T);
  ++T;
  T %= W;

  iLen = *((WORD*)sSize);

  if(iLen > S || 0 == iLen)
    return -1;

  // 패킷 복사 후 F 이동
}
```

3.1추상화 ILcNet: ILcNet 추상화 목적

- 설계 방향
  - Select / AsyncSelect / EventSelect / IOCP 사용법 다름
  - 하지만 응용 계층 입여기서는 같은 기능 필요
  - Create / Destroy
  - Listen / Connect
  - Send / Recv
  - FrameMove (Update)
  - 이 공통 사용 규칙을 ILcNet으로 추상화

- 응용 계층이 원하는 기능

```
Create
Listen / Connect
Send / Recv
FrameMove
Destroy

     ↓ 추상화

ILcNet
```

- 구현 내용
  - 순수 가상 함수
  - Network 사용 규칙
  - 구현 모델을 감추는 경계
  - AsyncSelect용 MsgProc까지 포함

```
interface ILcNet
{
  virtual int Create(...)=0;
  virtual void Destroy()=0;
  virtual int FrameMove()=0;
  virtual int Query(...)=0;
  virtual int Close()=0;
  virtual int Listen()=0;
  virtual int Connect(...)=0;
  virtual int Send(...)=0;
  virtual int Recv(...)=0;
  virtual LRESULT MsgProc(...)=0;
};
```

3.3 ILcNet 기능 단위 추상화

- 설계 방향
  - Create → 라이브러리 객체 생성과 준비
  - Listen → 서버 역할 시작
  - Connect → 클라이언트 역할 시작
  - Send → 패킷 송신 요청
  - Recv → 수신 패킷 꺼내기
  - FrameMove → 네트워크 I/O 갱신
  - MsgProc → AsyncSelect 메시지 처리

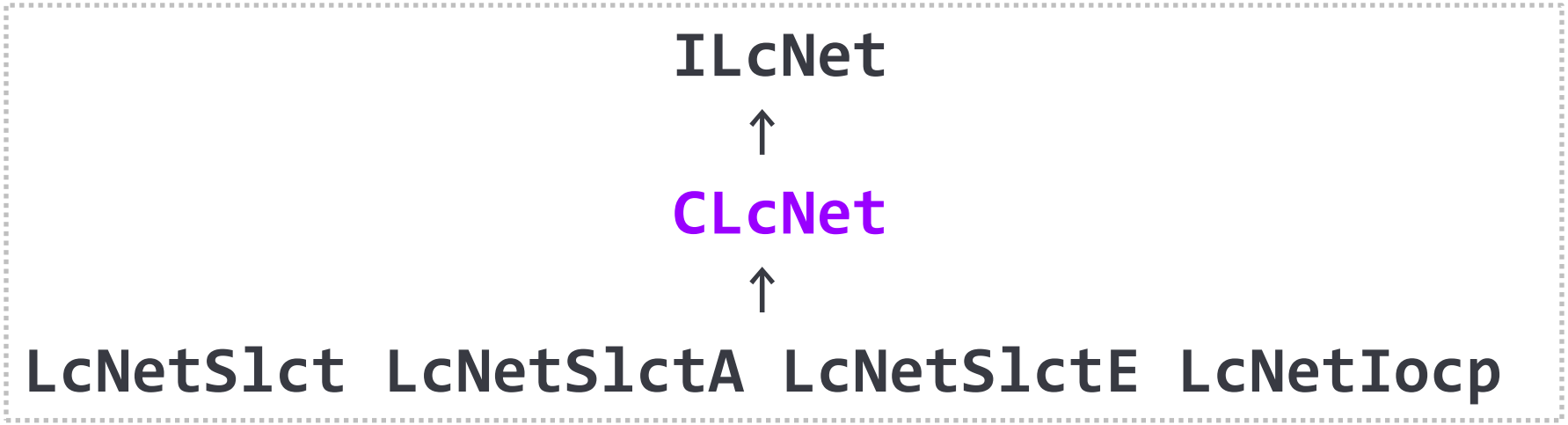
- 확인 사항
  - WinSock 함수 이름을 그대로 노출하지 않음
  - 네트워크 라이브러리 관점의 동작으로 추상화
  - 내부 구현은 I/O 모델별 클래스가 담당

- 설계 방향
  - 문자열로 구현 모델 선택
  - 생성 결과는 ILcNet*
  - 응용 계층은 실제 타입을 알 필요 없음
  - 테스트와 교체가 쉬움

```
int LcNet_CreateTcpServerEx(char* sCmd,
                ILcNet** pData,
                void* p1,   // IP
                void* p2,   // Port
                void* v1,   // Ex Value1
                void* v2);  // Ex Value2

int LcNet_CreateTcpClientEx(char* sCmd,
                ILcNet** pData,
                void* p1,
                void* p2,
                void* v1,
                void* v2);
```

- 설계 방향
  - ILcNet을 직접 구현하는 기본 클래스
  - 모든 모델이 공유하는 상태 보관
  - 기본 함수는 0 또는 -1 반환
  - 실제 구현은 파생 클래스에서 재정의
  - Thread wrapper 제공



- 구현 내용
  - 소켓 핸들
  - 서버/클라이언트 주소
  - IP / Port
  - 프로토콜 타입
  - 호스트 타입
  - 연결 상태

```
class CLcNet : public ILcNet
{
protected:
  SOCKET    m_scH    {};
  SOCKADDR_IN m_sdH {};

  std::string m_sIp;
  std::string m_sPt;

  int m_PtcType     {};
  int m_HstType     {};
  int m_nThConn     {};
};
```

4.3 CLcNet enum

- 구현 내용
  - 프로토콜 추상화
  - 호스트 역할 추상화
  - 이벤트 종류 추상화
  - 공용 상수 →
    파생 클래스에서 같이 사용

```
enum ELcNetPrtcl {
  NETPRT_NONE = 0,
  NETPRT_TCP = 1,
  NETPRT_UDP = 2,
};
enum ELcNetHstType {
  NETHST_NONE  = 0,
  NETHST_CLIENT = 1,
  NETHST_SERVER = 2,
};
enum ELcNetEventType {
  NETEVT_NONE = 0,
  NETEVT_SEND = 1,
  NETEVT_RECV = 2,
  NETEVT_ACCP = 4,
  NETEVT_CONN = 8,
  NETEVT_CLOSE = 16,
};
```

- 구현 내용
  - 공통 기반 클래스
  - 실제 I/O는 하지 않음
  - 인터페이스 구조 유지
  - 파생 클래스 구현 강제

```
int CLcNet::Create(void* p1, void* p2, void* p3,
void* p4)
{
  return 0;
}
void CLcNet::Destroy() {
}
int CLcNet::FrameMove() {
  return 0;
}
int CLcNet::Listen() {
  return 0;
}
int CLcNet::Connect(char* sIp, char* sPort) {
  return 0;
}
int CLcNet::Send(const char* pSrc, int iSnd,
        DWORD dMsg, SOCKET scH)
{
  return 0;
}
int CLcNet::Recv(char* pDst, int* iRcv,
        DWORD* dMsg, SOCKET* scH)
{
  return 0;
}
```

- 구현 내용
  - C 스타일 Thread 함수와 C++ 객체 연결
  - static 함수에서 객체 함수 호출
  - ProcRecv / ProcSend / ProcAccp / ProcWork 재정의
  - 스레드 기반 모델 구현의 공통 구조

```
DWORD WINAPI CLcNet::ThreadRecv(void* pParam) {
  return ((CLcNet*)pParam)->ProcRecv(pParam);
}
DWORD WINAPI CLcNet::ThreadSend(void* pParam) {
  return ((CLcNet*)pParam)->ProcSend(pParam);
}
DWORD WINAPI CLcNet::ThreadAccp(void* pParam) {
  return ((CLcNet*)pParam)->ProcAccp(pParam);
}
DWORD WINAPI CLcNet::ThreadWork(void* pParam) {
  return ((CLcNet*)pParam)->ProcWork(pParam);
}
```

5.1 구체화 2단계: Select 모델 구현 목적

- 설계 방향
  - 구조가 가장 직접적
  - FD_SET으로 소켓 목록 확인
  - FrameMove 안에서 send / recv 처리
  - 구현 예제 모델로 적합
  - AsyncSelect / EventSelect / IOCP 비교 기준

```
CLcNetSlct
      ↓
   FrameMove
      ↓
   select write → SendAllData
      ↓
   select read → accept / recv
```

- 구현 내용
  - CLcNet 상속
  - 클라이언트별 송수신 버퍼
  - 소켓 목록: m_rmF
  - 소켓 상태: m_rmH
  - Send 요청과
    실제 send 분리

```
class CLcNetSlct : public CLcNet
{
public:
  struct LcNetSlctHost
  {
    TRingBuf rbSnd;
    TRingBuf rbRcv;
  };

protected:
  TRingBuf m_rbSnd;
  TRingBuf m_rbRcv;

  FD_SET m_rmF;
  LcNetSlctHost m_rmH[FD_SETSIZE];
};
```

- 처리 과정
  - p1: IP
  - p2: Port
  - p3: Protocol
  - p4: Client / Server
  - TCP / UDP 결정
  - Client면 Connect
  - Server면 Listen

```
int CLcNetSlct::Create(...)
{
  auto sIp  = (const char*)p1;
  auto sPort = (const char*)p2;
  auto sPtcl = (const char*)p3;
  auto sSvr = (const char*)p4;
  if(sIp)
    m_sIp = sIp;
  if(sPort)
    m_sPt = sPort;
  if(0 == _stricmp("TCP", sPtcl))
    m_PtcType = NETPRT_TCP;
  else if(0 == _stricmp("UDP", sPtcl))
    m_PtcType = NETPRT_UDP;

  if(0 == _stricmp("Client", sSvr))
  {
    m_HstType = NETHST_CLIENT;
    return Connect(nullptr, nullptr);
  }
  else if(0 == _stricmp("Server", sSvr))
  {
    m_HstType = NETHST_SERVER;
    return Listen();
  }
  return -1;
}
```

- 주요 내용
  - 소켓 주소 설정은 LcNetUtil
  - 소켓 생성도 LcNetUtil
  - Connect / Bind / Listen도 LcNetUtil
  - CLcNetSlct는 흐름만 담당
  - 이것이 공통 함수 라이브러리화의 효과

5.5 Connect 흐름


```
Connect
  ↓
LcNet_SocketAddr
  ↓
LcNet_SocketTcpCreate / UdpCreate
  ↓
LcNet_SocketConnect
  ↓
LcNet_SocketNonBlocking
  ↓
LcNet_SocketNaggleOff
  ↓
Fd_Set
```

```
Listen 흐름
Listen
  ↓
LcNet_SocketAddr
  ↓
LcNet_SocketTcpCreate / UdpCreate
  ↓
LcNet_SocketBind
  ↓
LcNet_SocketListen
  ↓
Fd_Set
```


- 구현 내용
  - 응용 계층은 net->Send 호출
  - 내부에서는 PacketEncode
  - 송신 링버퍼에 저장
  - 실제 send는 FrameMove / SendAllData에서 처리
  - Send 요청과 실제 I/O 분리

5.7 Send 처리 과정


```
int CLcNetSlct::Send(const char* pSrc,
          int iSnd,
          DWORD dMsg,
          SOCKET scH)
{
  BYTE sBuf[PCK_BUF_MAX_MSG] = {0};
  int iSize = 0;
  int hr = 0;
  if(0xFFFFFFFF == scH)
  {
    iSize = LcNet_PacketEncode(sBuf,
                  (BYTE*)pSrc,
                  iSnd,
                  dMsg);

    hr = m_rbSnd.PushBack(sBuf, iSize);
    if(FAILED(hr))
      return -1;
  }
  else
  {
    // 대상 클라이언트 검색 후
    // m_rmH[nIdx].rbSnd에 PushBack
  }
  return 0;
}
```

응용 계층 Send

↓

PacketEncode

↓

rbSnd.PushBack

↓

SendAllData

↓

send

- 구현 내용
  - 응용 계층은 net->Recv 호출
  - 내부에서 rbRcv에서 패킷 Pop
  - PacketDecode 후 메시지 반환
  - 서버는 scH로 송신 클라이언트 확인
  - recv 시스템 호출과
    사용자 Recv 분리

5.8 Recv 처리 과정


```
int CLcNetSlct::Recv(char* pDst,
          int* iRcv,
          DWORD* dMsg,
          SOCKET* scH)
{
  BYTE sBuf[PCK_BUF_MAX_MSG] = {0};
  WORD iSize = 0;
  int hr = 0;
  *iRcv = 0;
  if(nullptr == scH)
  {
    hr = m_rbRcv.PopFront(sBuf, &iSize);
    if(FAILED(hr) || 0 == iSize)
      return -1;
    *iRcv = LcNet_PacketDecode(
                  (BYTE*)pDst,
                  dMsg,
                  sBuf,
                  iSize);
  }
  else
  {
    // 클라이언트별 rbRcv에서 PopFront
    // Decode 후 scH 반환
  }
  return 0;
}
```

recv 시스템 호출

↓

rbRcv.PushBack

↓

응용 계층 Recv

↓

rbRcv.PopFront

↓

PacketDecode

5.9 FrameMove(Update) 처리 과정

- 처리 과정
  - Send / Recv 함수 자체는 실제 I/O가 아님
  - FrameMove가 select로 상태 확인
  - 쓰기 가능하면 SendAllData
  - 읽기 가능하면 accept 또는 recv
  - 게임 루프에 넣기 좋은 구조

```
while(gameLoop)
{
  net->FrameMove();

  net->Send(...);
  net->Recv(...);
}
```

```
FrameMove
    ↓
FrameMoveCln or FrameMoveSvr
```

- 처리 과정
  - write 가능 검사
  - SendAllData 호출
  - read 가능 검사
  - listen socket이면 accept
  - client socket이면 recv
  - recv 결과 클라이언트 rbRcv 저장

```
FrameMoveSvr
   ↓
select write
   ↓
SendAllData
   ↓
select read
   ↓
listen socket?
   ├ yes → accept → Fd_Set
   └ no → recv → m_rmH[i].rbRcv.PushBack
```

- 처리 과정
  - write 가능 검사
  - SendAllData 호출
  - read 가능 검사
  - recv
  - 수신 데이터는 m_rbRcv에 저장
  - 응용 계층은 이후 Recv로 꺼냄

```
FrameMoveCln
   ↓
select write
   ↓
SendAllData
   ↓
select read
   ↓
recv
   ↓
m_rbRcv.PushBack
```

- 처리 과정
  - rbSnd에 쌓인 패킷을 실제 send
  - Client는 m_rbSnd 사용
  - Server는 클라이언트별 rbSnd 순회
  - Send 요청과 실제 send가 여기서 연결
  - 구현 관점에서는 send 부분 전송 주의

5.12 SendAllData


```
if(NETHST_CLIENT == m_HstType)
{
  if(FAILED(m_rbSnd.PopFront(sBuf, &iSize)))
    return 0;

  hr = send(m_scH, (char*)sBuf, iSize, 0);
}
else if(NETHST_SERVER == m_HstType)
{
  for(Uint j = 0; j < m_rmF.fd_count; ++j)
  {
    if(m_scH != m_rmF.fd_array[j])
    {
      if(SUCCEEDED(m_rmH[j].rbSnd.PopFront(sBuf, &iSize)))
      {
        hr = send(m_rmF.fd_array[j],
            (char*)sBuf,
            iSize,
            0);
      }
    }
  }
  return 0;
}
```

- 구현 내용
  - Select 모델은 소켓 목록 관리 필요
  - Fd_Set으로 추가
  - Fd_Clr로 제거
  - 제거 시 m_rmH도 함께 이동
  - 소켓 목록과 클라이언트 버퍼 배열 동기화 중요

```
m_rmF.fd_array
[listen][client1][client2][client3]

m_rmH
[unused][host1 ][host2 ][host3 ]
```

- 주요 내용
  - ILcNet 사용법은 동일
  - 내부 I/O 모델만 변경
  - Select: FrameMove polling
  - AsyncSelect: Window Message
  - EventSelect: WSAEVENT
  - IOCP: Completion Port

```
응용 계층
  ↓
ILcNet
  ↓
CLcNetSlct  : select
CLcNetSlctA : WSAAsyncSelect
CLcNetSlctE : WSAEventSelect
CLcNetIocp  : IOCP
```

- 구현 내용
  - Window Message 기반
  - HWND와 사용자 메시지 보유
  - MsgProc에서 FD_READ / FD_WRITE / FD_CLOSE 처리
  - WinAPI GUI 클라이언트에 적합
  - Send / Recv 구조는 동일하게 rbSnd / rbRcv 사용

```
class CLcNetSlctA : public CLcNet
{
protected:
  TRingBuf m_rbSnd;
  TRingBuf m_rbRcv;
  int m_iNsck;
  LcNetSlctHost m_rmH[WSA_MAXIMUM_WAIT_EVENTS];
  HANDLE m_hThSend;
  DWORD m_dThSend;
  int  m_nThSend;
  HWND m_hWnd;
  DWORD m_dwMsg;
};
```

- 구현 내용
  - WSAEVENT 배열 사용
  - Window Message 없이 이벤트 대기
  - Recv / Send Thread 사용
  - WSAEventSelect로 FD_ACCEPT / FD_READ / FD_WRITE / FD_CLOSE 등록
  - 서버/클라이언트 모두 이벤트 기반 처리 가능

```
class CLcNetSlctE : public CLcNet
{
protected:
  TRingBuf m_rbSnd;
  TRingBuf m_rbRcv;
  int m_iNsck;
  WSAEVENT m_rmE[WSA_MAXIMUM_WAIT_EVENTS];
  LcNetSlctHost m_rmH[WSA_MAXIMUM_WAIT_EVENTS];
  HANDLE m_hThRecv;
  DWORD m_dThRecv;
  int  m_nThRecv;
  HANDLE m_hThSend;
  DWORD m_dThSend;
  int  m_nThSend;
  WSAEVENT m_hEvnt;
  DWORD  m_dEvnt;
};
```

- 설계 방향
  - Overlapped I/O 기반
  - Completion Port로 완료 통지 수신
  - Worker Thread에서 완료 처리
  - Accept Thread에서 연결 수락
  - 클라이언트별 Overlap / RingBuffer 보유
  - 서버 확장성 중심

```
WSARecv 요청
  ↓
IOCP 완료 통지
  ↓
Worker Thread
  ↓
NETEVT_RECV
  ↓
RingBufPush
  ↓
AsyncRecv 재요청
```

- 구현 내용
  - 클라이언트 1개 상태
  - Recv / Send Overlap 분리
  - Recv / Send RingBuffer 분리
  - AsyncRecv / AsyncSend 함수 보유
  - Select의 m_rmH와 같은 역할을 더 고성능 구조로 확장

```
struct LcNetIocpHost
{
  SOCKET   scH;
  SOCKADDR_IN sdH;
  TlnOVERLAP olRcv;
  TlnOVERLAP olSnd;
  TRingBuf rbRcv;
  TRingBuf rbSnd;
  int AsyncRecv();
  int AsyncSend();
  void RingBufPush(int iRcv);
};
```

7. 정리: I/O 모델별 차이 정리

I/O 모델	I/O 확인 방식	중심 함수
Select	select 직접 호출	FrameMove
AsyncSelect	Window Message	MsgProc
EventSelect	WSAEVENT 대기	ProcRecv
IOCP	완료 통지	ProcWork

- **공통점**
  - ILcNet 사용
  - CLcNet 기반
  - LcNetUtil 사용
  - rbSnd / rbRcv 사용
  - PacketEncode / PacketDecode 사용
  - Send 요청과 실제 I/O 분리

- 주요 내용
  - 응용 계층 코드 단순화
  - I/O 모델 교체 가능
  - 공통 함수 중복 제거
  - 패킷 처리 규칙 통일
  - 송수신 버퍼 구조 통일
  - 구현 난이도 단계화 가능